

HDOC Day-34

C Program: Matrix Distributive Properties Using Variable-Length 2D Arrays

Aim: Write a menu-driven C program using separate functions to verify the left and right distributive properties of matrix multiplication.

Menu operations: 1) $A*(B+C) = (A*B) + (A*C)$, 2) $(A+B)*C = (A*C) + (B*C)$, 3) Exit.

Program:

```
#include <stdio.h>

void readMatrix(int r, int c, int a[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void addMatrix(int r, int c, int a[r][c], int b[r][c],
               int sum[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            sum[i][j] = a[i][j] + b[i][j];
}

void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1], int b[c1][c2],
                   int result[r1][c2])
{
    int i, j, k;
    for (i = 0; i < r1; i++) {
        for (j = 0; j < c2; j++) {
            result[i][j] = 0;
            for (k = 0; k < c1; k++)
                result[i][j] += a[i][k] * b[k][j];
        }
    }
}

int equalMatrix(int r, int c, int a[r][c], int b[r][c])
{
    int i, j;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            if (a[i][j] != b[i][j])
                return 0;
    return 1;
}

void printMatrix(int r, int c, int a[r][c])
{
    int i, j;
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++)
            printf("%d\t", a[i][j]);
        printf("\n");
    }
}

void leftDistributive(void)
{
    int m, n, p, i, j;
    printf("Enter m, n and p: ");
    scanf("%d%d%d", &m, &n, &p);

    int A[m][n], B[n][p], C[n][p];
    int BC[n][p], left[m][p];
    int AB[m][p], AC[m][p], right[m][p];

    printf("Enter matrix A (%d x %d):\n", m, n);
    readMatrix(m, n, A);
```

```

printf("Enter matrix B (%d x %d):\n", n, p);
readMatrix(n, p, B);
printf("Enter matrix C (%d x %d):\n", n, p);
readMatrix(n, p, C);

addMatrix(n, p, B, C, BC);
multiplyMatrix(m, n, p, A, BC, left);

multiplyMatrix(m, n, p, A, B, AB);
multiplyMatrix(m, n, p, A, C, AC);
addMatrix(m, p, AB, AC, right);

printf("\nA*(B+C):\n");
printMatrix(m, p, left);
printf("\n(A*B) + (A*C):\n");
printMatrix(m, p, right);

if (equalMatrix(m, p, left, right))
    printf("\nLeft distributive property is verified.\n");
else
    printf("\nProperty is not verified.\n");
}

void rightDistributive(void)
{
    int m, n, p;
    printf("Enter m, n and p: ");
    scanf("%d%d%d", &m, &n, &p);

    int A[m][n], B[m][n], C[n][p];
    int AB[m][n], left[m][p];
    int AC[m][p], BC[m][p], right[m][p];

    printf("Enter matrix A (%d x %d):\n", m, n);
    readMatrix(m, n, A);
    printf("Enter matrix B (%d x %d):\n", m, n);
    readMatrix(m, n, B);
    printf("Enter matrix C (%d x %d):\n", n, p);
    readMatrix(n, p, C);

    addMatrix(m, n, A, B, AB);
    multiplyMatrix(m, n, p, AB, C, left);

    multiplyMatrix(m, n, p, A, C, AC);
    multiplyMatrix(m, n, p, B, C, BC);
    addMatrix(m, p, AC, BC, right);

    printf("\n(A+B)*C:\n");
    printMatrix(m, p, left);
    printf("\n(A*C) + (B*C):\n");
    printMatrix(m, p, right);

    if (equalMatrix(m, p, left, right))
        printf("\nRight distributive property is verified.\n");
    else
        printf("\nProperty is not verified.\n");
}

int main(void)
{
    int choice;

    do {
        printf("\n--- MENU ---\n");
        printf("1. Verify A*(B+C) = (A*B) + (A*C)\n");
        printf("2. Verify (A+B)*C = (A*C) + (B*C)\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                leftDistributive();
                break;
            case 2:
                rightDistributive();
                break;
            case 3:
                printf("Program terminated.\n");
                break;
            default:

```

```
        printf("Invalid choice. Try again.\n");
    }
} while (choice != 3);

return 0;
}
```

Result: The program reads matrices of user-defined dimensions, performs matrix addition and multiplication using separate functions, and verifies both distributive properties.